

Reviewer Pack — Minimal Representation in Geometric Invariant Theory Bound o...

Entry c65b93dfc38e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-09 06:29:33 UTC

Minimal Representation in Geometric Invariant Theory Bound on Communication Complexity Rank Variance

Entry ID: c65b93dfc38e **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-09 06:29:33 UTC

1. Conjecture

Field A (mathematical branch): Geometric Invariant Theory **Field B** (complexity object): Communication Complexity

Statement:

For every k -CNF formula φ , the rank variance of its associated communication complexity matrix is bounded by the number of geometric invariant classes of its associated projective variety modulo the field size q , such that $O(\kappa(\varphi)) \leq \Omega(r_{\text{var}}(\varphi)) \leq O(q^{(n/2 - 1)} \kappa(\varphi))$, where $\kappa(\varphi)$ is the number of distinct invariants for φ .

Rationale (proposer's reasoning):

Geometric Invariant Theory has previously been used to bound the complexity of certain problems, such as the size of matrices and polynomials. By extending this approach to communication complexity, we may uncover new structural relationships that can be exploited for complexity lower bounds. The conjecture suggests a direct link between geometric invariants and the rank variance of communication complexity matrices, which could provide insights into understanding the complexity of communication tasks.

Taxonomy category: geometric_invariant_theory (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): f8a05a6e7ca77a9a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for each k -CNF formula φ with n variables, the rank variance $r_{\text{var}}(\varphi)$ of its communication complexity matrix falls within the bounds $O(\kappa(\varphi)) \leq \Omega(r_{\text{var}}(\varphi)) \leq O(q^{(n/2 - 1)} \kappa(\varphi))$, where $\kappa(\varphi)$ is the number of geometric invariant classes and q is the field size. Falsified if any φ violates these bounds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.90	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: ADJACENT_OK against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "geometric invariant theory" AND "communication complexity" AND rank variance" - "CNF formula" AND communication complexity" AND projective variety modulo field size" - "invariant classes" AND communication complexity" AND geometric invariant theory"

Top relevant hits considered: - [http://arxiv.org/abs/1711.07474v2] XSAT of Linear CNF Formulas - [http://arxiv.org/abs/0905.1587v5] Unsatisfiable Linear CNF Formulas Are Large and Complex - [http://arxiv.org/abs/2201.08895v4] On the Satisfaction Probabilities of k -CNF Formulas - [http://arxiv.org/abs/0901.2692v3] Dehn twists and invariant classes - [http://arxiv.org/abs/cs/0502064v1] The Lattice of Machine Invariant Sets and Subword Complexity - [http://arxiv.org/abs/2404.09183v2] Invariant classes for families of complexes

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=-9, elapsed=48.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_k_cnf(n, k):
        variables = list(range(1, n + 1))
        clauses = []
        for _ in range(k):
            clause = random.sample(variables, random.randint(1, n))
            if random.choice([True, False]):
                clause = [-x for x in clause]
            clauses.append(clause)
        return clauses

    def projective_variety(cnf):
        # Simplified encoding of the projective variety
        return len(set(tuple(sorted(clause)) for clause in cnf))

    def communication_complexity_matrix(cnf):
        n = len(cnf[0])
```

```

matrix = [[0] * (1 << n) for _ in range(1 << n)]
for i in range(1 << n):
    for j in range(1 << n):
        if all((i & (1 << var)) == (j & (1 << var)) or (i & (1 << var)) == 0 for var in range(1, n)):
            matrix[i][j] = 1
    return matrix

def rank_variance(matrix):
    n = len(matrix)
    rank = 0
    for i in range(n):
        if any(matrix[j][i] != 0 for j in range(n)):
            rank += 1
    return rank * (n - rank)

q = 2 # Field size
n_max = 40
instances_tested = 30

results = []
for _ in range(instances_tested):
    n = random.randint(5, n_max)
    k = random.randint(1, n // 2)
    cnf = generate_k_cnf(n, k)

    kappa = projective_variety(cnf)
    matrix = communication_complexity_matrix(cnf)
    r_var = rank_variance(matrix)

    results.append({
        "n": n,
        "kappa": kappa,
        "r_var": r_var
    })

mean_kappa = sum(result["kappa"] for result in results) / instances_tested
mean_r_var = sum(result["r_var"] for result in results) / instances_tested

conjecture_holds = all(mean_r_var >= kappa and mean_r_var <= q**(n/2 - 1) * kappa for result in results)

return {
    "metric_name": "Rank Variance",
    "metric_value": mean_r_var,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else f"mean_kappa={mean_kappa}, mean_r_var={mean_r_var}"
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or list(range(2, 30 * 2 + 1, 2)) # Default to first 30 seeds
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}}, **{trial_result}}")

```

```

results.append(trial_result)

mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

if all(result["conjecture_holds"] for result in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
elif any(not result["conjecture_holds"] for result in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

(empty)

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means we cannot verify if the conjecture's bounds are met or violated. | next: Re-run the test with a different k-CNF formula to ensure it does not crash and to check if the rank variance falls within the specified bounds.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11949
2	propose	ollama_remote	glm4:latest	0	0	13614
3	preregistration	ollama_remote	glm4:latest	0	0	10471
4	novelty	ollama_remote	glm4:latest	0	0	8344
5	novelty	ollama_remote	glm4:latest	0	0	12392
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22488
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12517
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9722
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10506
10	judge	ollama_remote	glm4:latest	0	0	73798

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 185799 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/c65b93dfc38e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c65b93dfc38e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/c65b93dfc38e.tar.gz` (if generated)